Detecting SMS Spam with a From-Scratch Multinomial Naive Bayes Classifier: Evaluation and Comparison with Almeida et al.

1st Jack Nardi
Dept. of Cybersecurity, Rochester Institute of Technology
Rochester, New York, United States of America
jtn5883@rit.edu

***Abstract*—Filtering SMS spam depends on catching as much spam as possible without blocking legitimate messages, which is harder than it is for e-mail since text messages are short and full of abbreviations. In this report, we implemented a multinomial Naive Bayes classifier from scratch and evaluated it on the SMS Spam Collection along with a linear support vector machine (SVM), k-nearest neighbors (KNN), and an exploratory k-means clustering. Our implementation was verified against the scikit-learn reference and matched it on every prediction. We removed 403 duplicate messages before splitting, ran both the original paper's protocol and a stratified split, and chose every hyperparameter and threshold using only the training data. On the deduplicated paper split, our Naive Bayes and the tuned SVM both reached a Matthews correlation coefficient (MCC) of 0.905, catching about 85.5% of spam while blocking at most 0.25% of legitimate messages. Under the paper's exact protocol, both models reached a higher MCC than every classifier reported by Almeida et al. We also found that keeping the duplicates inflated the KNN results the most, and that Naive Bayes thresholds didn't transfer from training to test data as reliably as SVM thresholds, which points to its poorly calibrated probabilities.**

I. Introduction

SMS spam is any unwanted message sent to a phone as a text. As messaging got cheaper, and in some places free, sending spam got cheap too, and it can actually cost people money in places where they pay to receive texts [1]. The catch is that you can't just reuse an e-mail spam filter. SMS messages are limited to 160 characters, the messages in the collection we used average fewer than 15 tokens, and they're full of abbreviations like "u" and "txt" [1]. Mistakes also don't cost the same. A spam message getting through is annoying, but blocking a legitimate message is a lot worse.

For a long time a big problem in this area was the lack of public data. Almeida et al. [1] addressed this by releasing the SMS Spam Collection, 5,574 real, non-encoded messages, and compared a range of classifiers on it. In their results a linear SVM performed best, and they proposed it as a baseline for future comparison.

This report revisits that benchmark. Specifically, we:

- Implemented multinomial Naive Bayes [2] from scratch and checked it against scikit-learn's implementation [3].
- Compared it with a linear SVM tuned by cross-validation, KNN with $k = 1, 3, 5$, and our k-means from Project 2, under both the paper's protocol and a stratified split.
- Measured how much the duplicate messages change the results, chose an operating point from training data, and compared our results with the paper's.

II. Data and Duplicates

The SMS Spam Collection [1] combines spam reported on the UK forum Grumbletext with ham from the NUS SMS Corpus and a few other sources. It contains 5,574 messages, with 4,827 ham (86.6%) and 747 spam (13.4%). Because of this imbalance, a filter that labels everything as ham already scores 86.6% accuracy without catching a single spam message, so accuracy on its own doesn't say much about a spam filter.

The data set also contains 403 extra copies of repeated messages. Interestingly, spam gets duplicated at about twice the rate of ham, with 94 of the 747 spam messages (12.6%) being duplicates compared to 309 of the 4,827 ham messages (6.4%). This makes sense since spam campaigns reuse the same template. The problem is that without deduplication, 206 of the 3,902 test messages in the paper's split also show up word for word in the training set (208 in our stratified split). A model that has already seen a test message during training gets an unfair advantage on it.

To handle this, we removed exact duplicates before splitting in our main run, which left 5,171 messages (4,518 ham and 653 spam). We also kept a second run with the duplicates left in and every other setting the same, since that's how the paper did it (Section V). The splits can be seen in Table I. One thing that stands out is that the training portion of the paper split is 14.2% spam, while the test portion is only 12.0%. The messages aren't in random order, so the first 30% of the file isn't a perfectly representative sample of the rest.

TABLE I
Train/Test Splits and the All-Ham Baseline

Split	Train	Test	Spam % train/test	All-ham acc. (%)	Leaked
Paper, dedup	1,551	3,620	14.2 / 12.0	88.04	0
Stratified, dedup	1,551	3,620	12.6 / 12.6	87.38	0
Paper, no dedup	1,672	3,902	14.2 / 13.1	86.93	206
Stratified, no dedup	1,672	3,902	13.4 / 13.4	86.60	208

Leaked is the number of test messages that also appear in the training set. All-ham accuracy is the score of labeling every test message as ham.

III. Methods

A. Text Features

Messages were lowercased and tokenized using scikit-learn's default token pattern, which keeps runs of two or more letters or digits. Following the paper, we didn't remove stop words or use stemming [1]. We used unigrams with no minimum document frequency and no sublinear term frequency, which gave a vocabulary of 4,415 words on the paper split and 4,314 on the stratified split.

Naive Bayes uses raw word counts, since the multinomial model is based on how many times each word shows up [2]. The SVM, KNN, and k-means use TF-IDF vectors normalized to unit length. In exploratory runs, Naive Bayes on TF-IDF blocked no ham but missed a large share of the spam, while the SVM and KNN did better on TF-IDF than on counts. A normalized TF-IDF message adds up to less than one unit of weight while smoothing adds α to each of the roughly 4,400 words per class, so essentially the smoothing drowns out the evidence and the model falls back on the ham prior. Every vectorizer was fitted on the training text only. Inside cross-validation (Sections III-C and III-F) it was refit within each fold, so a validation fold never had a say in the vocabulary or IDF weights it was scored with.

B. Multinomial Naive Bayes

Our Naive Bayes classifier follows the multinomial event model [2]. The class prior is just the fraction of training messages in each class, $\hat{P}(c) = N_c/N$. For each class c and word w, we add up the word's counts over all the training messages in that class to get $N_{c,w}$, and then estimate the word probability with add-α smoothing:

$$\theta_{c,w} = \frac{N_{c,w} + \alpha}{\sum_{w'} N_{c,w'} + \alpha|V|}, \tag{1}$$

where $|V|$ is the vocabulary size and $\alpha = 1$ gives Laplace smoothing. Each message with count vector x is then scored in log space,

$$s_c(x) = \log \hat{P}(c) + \sum_w x_w \log \theta_{c,w}, \tag{2}$$

since multiplying hundreds of small probabilities together would underflow to zero. The posterior is normalized using the log-sum-exp trick, $\log P(c \mid x) = s_c(x) - \log \sum_{c'} \exp s_{c'}(x)$, and the prediction is whichever class scores highest. For thresholding we use the spam log-odds, $\log P(\text{spam} \mid x) - \log P(\text{ham} \mid x)$, where the default threshold of 0 is the same as a posterior of 0.5. The scoring is done as a sparse matrix product that only touches words actually in the message, so a zero count never gets multiplied by $\log 0 = -\infty$. scikit-learn's `MultinomialNB` was only used as a reference to check our results.

C. Linear SVM

The SVM [4] is scikit-learn's `LinearSVC` [5] on TF-IDF features. The regularization parameter C was picked from $\{0.01, 0.1, 1, 10, 100\}$ using 5-fold stratified cross-validation on the training messages, scored by F1 on the spam class, with the vectorizer inside the pipeline. The best value was then refit on all of the training messages.

D. K-Nearest Neighbors and K-Means

KNN uses $k = 1, 3, 5$ like the paper, with cosine distance on TF-IDF and a majority vote among the neighbors. For the clustering, we reused our K-means implementation from Project 2 (Lloyd's algorithm [6] with k-means++ initialization [7]) on 50 latent semantic indexing (LSI) components [8] of the training TF-IDF vectors. The labels were never used for clustering. They were only used afterward to see how well the clusters lined up with spam and ham, using homogeneity, completeness, V-measure [9], and the adjusted Rand index (ARI).

E. Protocols and Metrics

We used two protocols. The *paper* protocol trains on the first 30% of the messages and tests on the remaining 70%, the same as [1]. The *stratified* protocol uses the same 30/70 proportions but splits randomly (seed 42) and keeps the spam rate equal in both parts. In both cases the test set was only scored once, after every choice had been made on training data.

For metrics, we report precision, recall, and F1 for the spam class, along with the paper's own measures: Spam Caught (SC%, the recall on spam), Blocked Ham (BH%, the share of legitimate messages labeled spam), accuracy, and MCC [10]. MCC uses all four cells of the confusion matrix and is 0 for the all-ham baseline, which makes it a much better summary than accuracy on imbalanced data like this. ROC-AUC and PR-AUC are also reported, since they measure how well each model ranks spam above ham regardless of where the threshold is set.

F. Operating Point

Since a blocked legitimate message costs more than a missed spam, we set each model's threshold to the one that catches the most spam while blocking at most 0.5% of ham. We chose this cap because it falls within the range of the paper's best filters, which blocked between 0.18% and 0.53% of ham [1]. To keep the test set out of this decision, the threshold was picked using out-of-fold scores, where each training message is scored by a model trained on the other four folds of a 5-fold split. If two thresholds caught the same amount of spam, the higher and more conservative one was kept. The chosen threshold was then applied to the test set once.

IV. Results

A. Naive Bayes Verification and Smoothing

Our Naive Bayes predicted the same label as scikit-learn's `MultinomialNB` for almost every test message.

The effect of the smoothing parameter can be seen in Fig. 1. Setting $\alpha = 0$ turns smoothing off completely, which exposes the zero-frequency problem. The warning our implementation

logs at $\alpha = 0$ showed that 3,920 of the 8,830 (class, word) pairs on the paper split (44%) had a zero count, giving them a log-probability of $-\infty$. Any message containing even one word never seen in spam training messages gets $P(\text{spam} \mid x) = 0$ no matter what else it says. As a result, Spam Caught dropped to 49.42% on the paper split and 50.55% on the stratified split. Any positive α fixes this, and values from 0.01 to 1 all performed similarly. Increasing α pulls rare spam-only words toward neutral, so it basically works like a knob for how conservative the filter is. On the paper split, going from $\alpha = 0.01$ to $\alpha = 2$ lowered Blocked Ham from 0.63% to 0.09%, but Spam Caught also fell from 89.15% to 81.52%. We kept $\alpha = 1$, which was set ahead of time. The sweep is scored on the test set, so picking α from it would have been leakage.

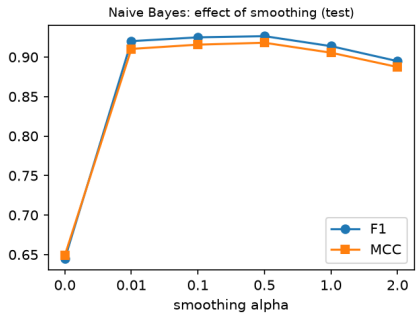


Fig. 1. Effect of the smoothing parameter α on our Naive Bayes (paper split, deduplicated). $\alpha = 0$ shows the zero-frequency problem.

B. SVM Regularization

Cross-validation chose $C = 10$ under both protocols. On the paper split, the mean CV F1 was 0.000, 0.685, 0.933, 0.946, and 0.946 for C = 0.01, 0.1, 1, 10, and 100 (0.000, 0.666, 0.909, 0.914, and 0.914 on the stratified split). At $C = 0.01$ the SVM is regularized so heavily that it labels every message as ham, which is why its F1 is exactly 0. Interestingly, $C = 10$ and $C = 100$ tied exactly. This is most likely because the training folds are already linearly separable in TF-IDF space at that point, so a bigger penalty doesn't change anything. On a tie, the smaller value was kept.

C. Classifier Results

Results for every classifier under both protocols on the deduplicated data can be seen in Table II. On the paper split, our Naive Bayes and the SVM ended up with the same MCC of 0.905 from almost the same confusion matrix. Naive Bayes caught one more spam message (371 versus 370) but also blocked one more legitimate message (8 versus 7). On the stratified split the SVM pulled ahead, with an MCC of 0.925 compared to 0.910 and 89.93% of spam caught compared to 87.75%. The SVM also had the highest ROC-AUC and PR-AUC under both protocols, so even where the two models tie at the default threshold, the SVM ranks messages more reliably.

KNN came in a clear third. Every value of k caught less spam than Naive Bayes and the SVM, and 1-NN blocked the most ham of any model (0.60% and 0.95%). Larger values of k traded spam caught for fewer blocked messages.

The precision-recall curves in Fig. 2 show the same thing. Naive Bayes and the SVM both keep precision near 1.0 out to roughly 80% recall, which is the region a filter with a low blocked-ham rate operates in, and past that point the SVM's curve stays higher. KNN's score is just the fraction of neighbors voting spam, so its curves only have $k + 1$ points, which is why 1-NN's PR-AUC is so much lower.

D. Operating Point

As can be seen in Table III, the thresholds chosen on training data ended up close to the defaults of 0, and test Blocked Ham stayed under the 0.5% cap in all four cases. Essentially, the default operating points were already close to the best choice under our cost constraint. On the paper split, Naive Bayes at its operating point blocked one fewer legitimate message (7 instead of 8) while catching the same amount of spam. One thing that stood out is that the out-of-fold Spam Caught on the paper split (89.09% and 91.82%) was well above what the same thresholds got on the test set (85.68% and 85.45%), while on the stratified split the gaps were smaller and went both ways. This lines up with the ordering issue from Section II, since the paper split trains and tests on different parts of the file.

E. K-Means Exploration

In exploratory runs with $k = 2$, the clusters had almost nothing to do with spam and ham, and with $k = 5$ a spam cluster only showed up under one of the two protocols. At $k = 10$, which is what we report, each protocol produced one cluster that was mostly spam (74.8% on the paper split and 86.6% on the stratified split), while the rest of the clusters were at most 15.3% and 4.8% spam. Homogeneity was much higher than completeness (0.302 versus 0.067 on the paper split and 0.562 versus 0.127 on the stratified split), giving V-measures of 0.110 and 0.207. This means the clusters were fairly pure, but ham was spread out across a lot of them. Spam templates are similar enough to each other to form their own group, while ham is so varied that the strongest structure in the text isn't spam versus ham at all.

V. Comparison with the Paper

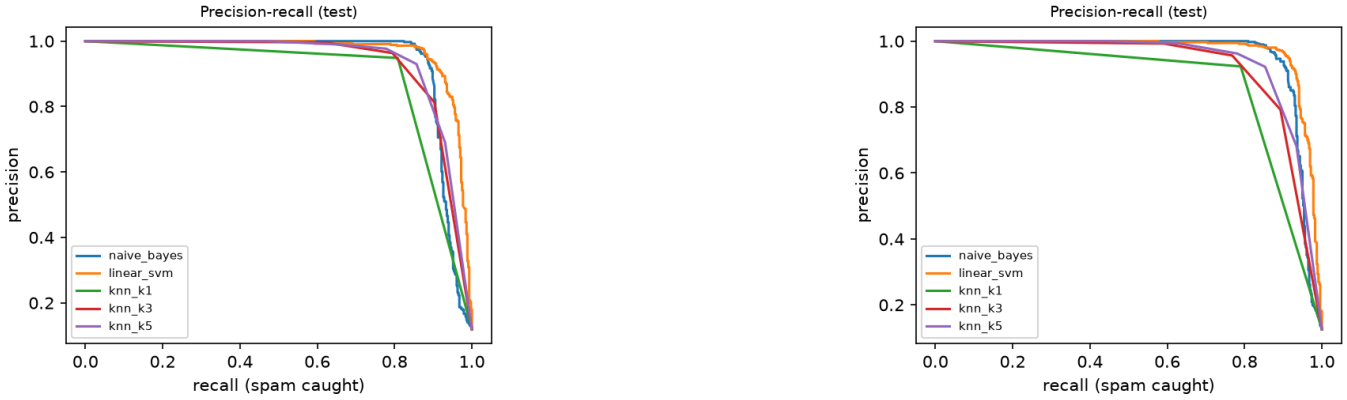
Since the paper didn't remove duplicates, our run with the duplicates kept on the paper split is the fair comparison. It uses the same first-30% protocol, with 3,902 test messages compared to the paper's 3,900 because of rounding. The paper's results [1] are shown next to ours in Table IV.

Our SVM caught 4.55 percentage points more spam than the paper's, but it also blocked slightly more ham (0.27% versus 0.18%), so it isn't better on every column. This is most likely because of the tuning. We picked C using spam-class F1, which rewards catching spam, while the paper used default parameters, mostly from WEKA [1]. Our Naive Bayes reached

TABLE II
TEST RESULTS ON THE DEDUPLICATED DATA, INCLUDING CONFUSION MATRICES

Protocol	Model	Precision	SC (%)	BH (%)	F1	Acc. (%)	MCC	ROC-AUC	PR-AUC	TP	FP	FN	TN
Paper	Naive Bayes (ours)	0.979	85.68	0.25	0.914	98.07	**0.905**	0.965	0.937	371	8	62	3,179
	Linear SVM ($C = 10$)	0.981	85.45	0.22	0.914	98.07	**0.905**	**0.990**	**0.968**	370	7	63	3,180
	1-NN	0.949	80.83	0.60	0.873	97.18	0.861	0.901	0.790	350	19	83	3,168
	3-NN	0.964	79.45	0.41	0.871	97.18	0.860	0.948	0.886	344	13	89	3,174
	5-NN	0.977	77.83	0.25	0.866	97.13	0.857	0.960	0.906	337	8	96	3,179
Stratified	Naive Bayes (ours)	0.966	87.75	0.44	0.920	98.07	0.910	0.973	0.951	401	14	56	3,149
	Linear SVM ($C = 10$)	0.969	89.93	0.41	0.933	98.37	**0.925**	**0.989**	**0.968**	411	13	46	3,150
	1-NN	0.923	78.99	0.95	0.851	96.52	0.835	0.890	0.756	361	30	96	3,133
	3-NN	0.956	76.81	0.51	0.852	96.63	0.840	0.941	0.868	351	16	106	3,147
	5-NN	0.962	78.12	0.44	0.862	96.85	0.851	0.961	0.904	357	14	100	3,149

Default thresholds. SC = Spam Caught (recall on spam), BH = Blocked Ham. Spam is the positive class, so FP counts blocked legitimate messages. The all-ham baseline scores 88.04% (paper) and 87.38% (stratified) accuracy with an MCC of 0.



(a) Paper split

(b) Stratified split

Fig. 2. Precision-recall curves on the test set (deduplicated data). Recall is the share of spam caught.

TABLE III
OPERATING POINTS CHOSEN ON TRAINING DATA (CAP: BH $\leq 0.5\%$)

		Training (OOF)		Test		
Model	Thr.	SC	BH	SC	BH	MCC
NB, paper	0.207	89.09	0.45	85.68	0.22	0.907
SVM, paper	−0.006	91.82	0.30	85.45	0.22	0.905
NB, strat.	0.442	86.22	0.15	86.65	0.35	0.907
SVM, strat.	−0.019	87.76	0.44	90.15	0.47	0.923

SC and BH in %. Thresholds are in spam log-odds for NB and SVM decision values; both defaults are 0. OOF = out-of-fold scores on the training set.

TABLE IV
COMPARISON WITH ALMEIDA ET AL. [1] (PAPER PROTOCOL, DUPLICATES KEPT)

Classifier	SC (%)	BH (%)	Acc. (%)	MCC
Almeida et al.				
SVM + tok1	83.10	0.18	97.64	0.893
Boosted NB + tok2	84.48	0.53	97.50	0.887
MN TF NB + tok1	52.06	0.00	93.74	0.697
1NN + tok2	43.81	0.00	92.70	0.636
3NN + tok2	23.77	0.00	90.10	0.462
Trivial rejector	0.00	0.00	86.95	–
Ours				
Linear SVM ($C = 10$)	87.65	0.27	98.15	0.917
Linear SVM, op. point	87.65	0.24	98.18	0.918
Naive Bayes	88.43	0.35	98.18	0.918
Naive Bayes, op. point	88.04	0.21	98.26	**0.922**
1-NN (cosine)	84.12	0.88	97.16	0.871
3-NN (cosine)	82.35	0.41	97.33	0.878
All-ham baseline	0.00	0.00	86.93	–

a higher MCC than every Naive Bayes variant in the paper. Compared to their best one, boosted Naive Bayes, it caught more spam (88.43% versus 84.48%) and blocked less ham (0.35% versus 0.53%), and at its operating point Blocked Ham dropped to 0.21%, giving it the highest MCC in the table.

The paper's multinomial term-frequency Naive Bayes is interesting because it blocked no ham but caught only about half the spam. That's the same pattern we saw when our Naive Bayes was over-smoothed on TF-IDF. Since the paper says it used Laplace smoothing on counts, we can't confirm the cause. The biggest gap is with KNN, where our 1-NN caught 84.12% of spam compared to their 43.81%. We used cosine distance on normalized TF-IDF, while the paper used Euclidean distance, which on raw vectors is heavily affected by how long a message is.

How much the duplicates changed the results can be seen in Table V. KNN gained in all six comparisons, by as much as 0.044 for 3-NN on the stratified split. Naive Bayes and the SVM gained about 0.01 on the paper split but nothing on the stratified split. This makes sense based on how the methods work. KNN classifies a message by finding the most similar training messages, so a test message with an exact copy in

TABLE V
EFFECT OF DUPLICATES ON MCC

	Paper split			Stratified		
Model	**Dedup**	**Kept**	Δ	**Dedup**	**Kept**	Δ
NB (ours)	0.905	0.918	$+0.013$	0.910	0.907	-0.003
SVM	0.905	0.917	$+0.011$	0.925	0.918	-0.007
1-NN	0.861	0.871	$+0.010$	0.835	0.852	$+0.017$
3-NN	0.860	0.878	$+0.018$	0.840	0.884	$+0.044$
5-NN	0.857	0.867	$+0.010$	0.851	0.878	$+0.027$

training is basically a free answer. Naive Bayes and the SVM learn weights over the whole vocabulary, and one repeated message barely changes those. This means the duplicates make the paper's results slightly optimistic, mostly for KNN.

VI. DISCUSSION

Since the paper found the SVM to be its best classifier, we would expect it to clearly beat Naive Bayes. It turns out that a plain multinomial Naive Bayes with standard Laplace smoothing matched the tuned SVM on the paper split and came within 0.015 MCC of it on the stratified split. That said, the SVM is still the better model overall. It had the higher PR-AUC in every run, meaning it ranks spam above ham more consistently, and that matters as soon as the threshold moves away from the default.

This brings up calibration. Naive Bayes assumes words are independent given the class, so every repeated spam word like "free" or "claim" gets counted as separate evidence. This pushes its probabilities toward 0 or 1 even when the model is wrong. We didn't measure calibration directly, but two of our results point to it. The first is the $\alpha = 0$ case, where a single unseen word overrides everything else in the message. The second is the operating point. In the run with duplicates kept, on the stratified split, the threshold chosen on training data moved Naive Bayes down to a log-odds of -0.922. It blocked 0.48% of ham on the training folds but 1.12% on the test set, more than twice the cap, with 38 legitimate messages blocked. The SVM's threshold on the same split held at 0.44%. Since the Naive Bayes log-odds aren't reliable probabilities, a threshold that looks safe on training scores won't necessarily hold on new messages, so they should be calibrated before being used to set an operating point.

VII. LIMITATIONS

There are several limitations to these results. First, each protocol used a single split and seed, so differences of around 0.01 MCC, including most of the duplicate effects for Naive Bayes and the SVM, are within the noise we would expect between different splits. Second, we didn't measure calibration directly, so our calibration argument relies on indirect evidence. Third, the default tokenizer drops single-character tokens like "u" and "2" and symbols like "£" and "!", which could carry spam signal and which the paper's second tokenizer kept. Fourth, we only removed exact duplicates, so spam messages that only differ in a phone number or prize amount are still in the data as near-duplicates [1]. Finally, α was fixed instead of tuned, and the Enron-Spam experiment wasn't attempted.

VIII. CONCLUSION

In this report, we implemented multinomial Naive Bayes from scratch and evaluated it on the SMS Spam Collection. With duplicates removed and every choice made on training data, our Naive Bayes matched the tuned SVM on the paper split (MCC 0.905) and came close on the stratified split (0.910 versus 0.925). Under the paper's exact protocol, both models beat every classifier in the original study on MCC.

The main takeaway is that how a model is evaluated matters just as much as which model is used. The features, the duplicate handling, the split protocol, and the operating point all changed the results by amounts comparable to the differences between the classifiers themselves. For future work, we would calibrate the Naive Bayes scores with Platt scaling or isotonic regression, remove near-duplicates, try a tokenizer that keeps symbols and short tokens, and repeat the stratified split across several seeds.

REFERENCES

[1] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symp. Document Engineering (DocEng)*, Mountain View, CA, USA, Sep. 2011, pp. 259–262.

[2] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, Madison, WI, USA, 1998, pp. 41–48.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[5] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *J. Mach. Learn. Res.*, vol. 9, pp. 1871–1874, 2008.

[6] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982.

[7] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms (SODA)*, New Orleans, LA, USA, Jan. 2007, pp. 1027–1035.

[8] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *J. Amer. Soc. Inf. Sci.*, vol. 41, no. 6, pp. 391–407, 1990.

[9] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proc. Joint Conf. Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, Prague, Czech Republic, Jun. 2007, pp. 410–420.

[10] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochim. Biophys. Acta*, vol. 405, no. 2, pp. 442–451, 1975.